

Wavetable Operator Validation Guide

This guide is for **human listening validation** of the current `vivid-wavetable` package.

It is not the main synth-building tutorial. If you want the beginner path for learning the package, start with [README.md](#). This guide is for the moment when you need to answer a narrower question:

- does the audible voice path behave correctly?
- is this a wiring mistake, an operator problem, or a higher-level voice problem?
- does the shipped module or instrument still sound like the role it is supposed to play?

Automated regression checks already exist for package tests, smoke graphs, modules, assets, and instrument metadata. Those run outside this document. This guide is only for the things that still need human ears.

The validation surface in this document is the **full user-facing listening path**:

- package operators:
 - `PolyVoiceAllocator`
 - `WavetableOsc`
 - `VoiceMixer`
 - `VoiceDrive`
 - `SubOsc`
 - `AnalogOsc`
 - `NoiseLayer`
- core operators that matter to shipped voices:
 - `EnvelopeAu`
 - `Filter`
 - `DualFilter`
- expressive input behavior where shipped instruments depend on it

The goal is not only to prove that one operator can make sound. The goal is to prove that the package's audible, playable voice path still behaves the way the current April 4 package intends.

What Lanes Mean

In `vivid-wavetable`, one note becomes one lane.

- `PolyVoiceAllocator` turns notes into per-note lanes
- `WavetableOsc`, `AnalogOsc`, `SubOsc`, `NoiseLayer`, and `VoiceDrive` stay per-note
- `EnvelopeAu` should usually be per-note in these graphs: `voices/gates` → `env/gate`
- `VoiceMixer` is the point where separate note lanes get reduced to normal stereo output

Think of it like this:

- before `VoiceMixer`, each note is still its own thing
- after `VoiceMixer`, the notes are combined into one stereo signal

Global vs per-note:

- global: `ClockAu`, chord timing, synced LFO timing, macro movement
- per-note: oscillator pitch, gate, velocity, lane ids, note envelopes, note-shaped filter motion, per-voice drive, note-following sub/noise layers

How To Use This Guide

For each stage:

1. Create the nodes exactly as listed.
2. Make the connections exactly as listed.
3. Set the key params exactly as listed.
4. Listen for the expected result.
5. If it does not match, use the failure notes before moving on.

Use a fresh graph for Stage 1, then keep building on it through Stage 7. After that, make small comparison graphs or load the shipped reference graphs for the later validation passes.

Use this guide when you want answers to questions like:

- is this operator actually working?
- is this graph wired correctly for lanes?
- is this envelope really per-note?
- is this drive or filter block acting per-note instead of globally?
- does this module still sound like the role it is meant to fill?
- does builtin vs custom wavetable selection still sound intentional?

Stage 1: Silent Oscillator Sanity

Build this graph:

- WavetableOsc named osc
- audio_out named out

Connect:

```
osc/output -> out/input
```

Set these params on osc:

- amplitude = 0.25
- wavetable_source = Builtin
- wavetable_family = AnalogWarm
- wavetable_member = Core
- position = 0.35

What you should hear:

- silence

What you should see:

- a graph with only two nodes and one connection
- no lane sources yet

Why this is correct:

- the oscillator should not self-start
- it needs lane inputs like frequency and gate before it should make sound

If it sounds wrong:

- if you hear any pitched sound or hiss, the oscillator is not respecting lane or gate input

Operator checklist:

- WavetableOsc: stays silent without note lanes

Stage 2: Add PolyVoiceAllocator

Add:

- ClockAu named clock
- ChordProgressionAu named chords
- PolyVoiceAllocator named voices

Connect:

```
clock/beat_phase -> chords/beat_phase
chords/notes -> voices/notes_in
chords/velocities -> voices/velocities_in
chords/gates -> voices/gates_in

voices/frequencies -> osc/frequencies
voices/gates -> osc/gates
voices/velocities -> osc/velocities
voices/lane_ids -> osc/lane_ids
```

Set these params:

- clock/bpm = 96
- voices/max_voices = 6
- chords/degree_0 = 0
- chords/degree_1 = 3
- chords/degree_2 = 4
- chords/degree_3 = 5

What you should hear:

- a rough, raw, changing chord sound
- clearly more than one note at once
- no polish yet

What you should see:

- PolyVoiceAllocator sitting between the chord source and the oscillator
- all four lane connections from voices into osc

Why this is correct:

- this stage proves the allocator is creating note lanes and the oscillator is reading them

If it sounds wrong:

- one note only: lane allocation is not working
- stuck pitch: gate or lane id flow is wrong
- silence: check all voices -> osc connections first

Operator checklist:

- PolyVoiceAllocator: produces frequencies, gates, velocities, lane_ids

- Wavetable0sc: responds to lane-driven notes

Stage 3: Add VoiceDrive Before Reduction

Add:

- VoiceDrive named drive

Rewire:

```
osc/output -> drive/input
voices/velocities -> drive/velocities
drive/output -> out/input
```

Remove or disconnect:

```
osc/output -> out/input
```

Set these params:

- drive/drive = 0.18
- drive/tone = 0.52
- drive/mix = 0.55
- drive/output_level = 1.0
- drive/velocity_to_drive = 0.25

What you should hear:

- the same rough polyphonic note behavior as Stage 2
- more body and harmonic edge
- more response on harder notes, not one smeared global distortion wash

What you should see:

- VoiceDrive still on the per-note side of the graph
- voices/velocities driving drive/velocities

Why this is correct:

- VoiceDrive belongs before VoiceMixer
- it should preserve note lanes while adding body and glue

If it sounds wrong:

- the whole patch sounds like one post-mix saturator: the drive is in the wrong place
- note attacks stop reading separately: lane behavior is being lost
- nothing changes: check osc/output -> drive/input and the drive mix amount

Operator checklist:

- VoiceDrive: adds per-note body without collapsing the lane structure

Stage 4: Add VoiceMixer and Per-Note Amp Envelope

Add:

- VoiceMixer named mixer
- EnvelopeAu named amp_env

Replace the old output path:

```
drive/output -> mixer/input
amp_env/value -> mixer/amp_env_audio
voices/velocities -> mixer/velocities
mixer/output -> out/input
voices/gates -> amp_env/gate
```

Remove or disconnect:

```
drive/output -> out/input
```

Set these params:

- `amp_env/attack = 0.01`
- `amp_env/decay = 0.25`
- `amp_env/sustain = 0.70`
- `amp_env/release = 0.40`
- `mixer/stereo_spread = 0.70`

What you should hear:

- clear chord changes
- cleaner note attacks and releases
- a normal stereo signal instead of raw lane chaos

What you should see:

- `amp_env` driven from `voices/gates`
- `VoiceMixer` taking per-note audio and producing the final stereo signal

Why this is correct:

- this is the first proper polysynth-style graph
- `VoiceMixer` is where separate notes stop being separate

If it sounds wrong:

- first chord loud, later chords soft: the envelope is probably not really per-note
- silence: check `amp_env/value -> mixer/amp_env_audio`
- still raw and chaotic: you are probably still bypassing `VoiceMixer`

Operator checklist:

- `VoiceMixer`: reduces per-note audio to stereo
- `EnvelopeAu`: works as a per-note amp envelope when driven from `voices/gates`

Stage 5: Add Filter With Per-Note Envelope

Add:

- Filter named `filter`
- `EnvelopeAu` named `filt_env`

Rewire:

```
drive/output -> filter/input
filter/output -> mixer/input
```

```
voices/frequencies -> filter/frequencies
voices/gates -> filt_env/gate
filt_env/value -> filter/cutoff_mod
```

Remove or disconnect:

```
drive/output -> mixer/input
```

Set these params:

- filter/mode = LowPass
- filter/cutoff = 2200
- filter/resonance = 0.18
- filt_env/attack = 0.02
- filt_env/decay = 0.50
- filt_env/sustain = 0.20
- filt_env/release = 0.35

What you should hear:

- each note or chord attack should open the filter briefly
- higher notes should feel naturally brighter because the filter is tracking note frequency

What you should see:

- voices/frequencies -> filter/frequencies
- voices/gates -> filt_env/gate
- filt_env/value -> filter/cutoff_mod

Why this is correct:

- this proves the simple filter path is behaving properly in a lane-aware graph
- the envelope should shape each note, not the whole patch globally

If it sounds wrong:

- big global swell instead of per-note movement: wrong envelope wiring
- dull and disconnected pitch response: missing voices/frequencies -> filter/frequencies

Operator checklist:

- Filter: tracks note pitch correctly in a poly graph
- EnvelopeAu: can shape note-specific filter motion

Stage 6: Verify Wavetable Source Selection

Stay on the same graph. Change only the Wavetable0sc source.

Test these combinations one at a time:

1. AnalogWarm / Core
2. BrightDigital / Glass
3. VocalFormant / Rich
4. TextureMotion / Air

Keep the rest of the graph unchanged.

What you should hear:

- AnalogWarm / Core: rounder, more familiar, less glassy
- BrightDigital / Glass: brighter, sharper, more crystalline
- VocalFormant / Rich: more vowel-like or hollow vocal color
- TextureMotion / Air: more airy and synthetic, less conventional

What you should see:

- only the source selection changing
- no rewiring needed

Why this is correct:

- the families should sound meaningfully different even in the same simple graph

If it sounds wrong:

- everything sounds nearly the same: source selection may not be applied
- one family goes silent while the others work: family/member lookup may be broken

Operator checklist:

- Wavetable0sc: family/member selection audibly changes the source

Stage 7: Verify Wavetable Motion Inputs

Add:

- LfoAu named pos_lfo
- optional EnvelopeAu named warp_env

Connect:

```
pos_lfo/value -> osc/position_mod_audio
voices/gates -> warp_env/gate
warp_env/value -> osc/warp_mod_audio
```

Set these params:

- pos_lfo/frequency = 0.18
- pos_lfo/amplitude = 0.18
- warp_env/attack = 0.01
- warp_env/decay = 0.35
- warp_env/sustain = 0.00
- warp_env/release = 0.10
- osc/warp_mode = one clearly audible mode
- osc/warp_amount = 0.20

What you should hear:

- position_mod_audio: slow timbre travel, not pitch wobble
- warp_mod_audio: note attacks should change shape or bite

What you should see:

- motion targets going to position_mod_audio and warp_mod_audio, not pitch_mod_audio

Why this is correct:

- wavetable position should move tone color
- warp should move shape character

If it sounds wrong:

- pitch-like wobble means you may be modulating the wrong input
- no change at all means the modulation amount or target is wrong

Operator checklist:

- WavetableOsc: position and warp modulation affect timbre, not note pitch

Stage 8: Verify AnalogOsc

Build a second small graph or duplicate the Stage 4 graph and swap the oscillator.

Use:

- AnalogOsc named analog
- PolyVoiceAllocator
- EnvelopeAu
- VoiceMixer
- audio_out

Connect:

```
voices/frequencies -> analog/frequencies
voices/gates -> analog/gates
voices/velocities -> analog/velocities
voices/lane_ids -> analog/lane_ids
analog/output -> mixer/input
voices/gates -> amp_env/gate
amp_env/value -> mixer/amp_env_audio
voices/velocities -> mixer/velocities
mixer/output -> out/input
```

Test a few waveforms.

What you should hear:

- simpler, more classic shapes than the wavetable oscillator
- clear differences between waveforms

What you should see:

- the same lane structure as the wavetable graph

Why this is correct:

- AnalogOsc should behave like another per-note source, not a different graph model

If it sounds wrong:

- if it feels like a pre-summed mono bus before VoiceMixer, lane behavior is wrong

Operator checklist:

- AnalogOsc: works like a proper per-note source in the same lane architecture

Stage 9: Verify SubOsc

Add SubOsc under either WavetableOsc or AnalogOsc.

Add:

- SubOsc named sub
- second VoiceMixer named mix_sub

Connect:

```
voices/frequencies -> sub/frequencies
voices/gates -> sub/gates
voices/velocities -> sub/velocities
voices/lane_ids -> sub/lane_ids
sub/output -> mix_sub/input
amp_env/value -> mix_sub/amp_env_audio
voices/velocities -> mix_sub/velocities
```

Then mix it with your main layer using a normal stereo Mixer.

Set these params:

- sub/level = 0.25
- mix_sub/stereo_spread = 0.10

What you should hear:

- more low support and body
- not a separate melody or detached drone

What you should see:

- SubOsc getting the full lane set, just like the main oscillator

Why this is correct:

- the sub is supposed to reinforce each note, not float separately under the whole patch

If it sounds wrong:

- detached bass drone: note lanes are not being followed properly
- no clear low support: sub level, envelope, or mixer path is wrong

Operator checklist:

- SubOsc: tracks each note lane and reduces correctly through VoiceMixer

Stage 10: Verify NoiseLayer

Add:

- NoiseLayer named noise
- third VoiceMixer named mix_noise

Connect:

```
voices/frequencies -> noise/frequencies
voices/gates -> noise/gates
voices/velocities -> noise/velocities
```

```
voices/lane_ids -> noise/lane_ids
noise/output -> mix_noise/input
amp_env/value -> mix_noise/amp_env_audio
voices/velocities -> mix_noise/velocities
```

Test two cases.

Case 1: airy sustained layer

- level = 0.06
- tone = 0.68
- attack_burst = 0.08
- attack_decay_ms = 45

Expected sound:

- openness and breath
- not constant global hiss

Case 2: attack-heavy articulation layer

- level = 0.10
- tone = 0.82
- attack_burst = 0.60
- attack_decay_ms = 25

Expected sound:

- stronger pick or breath at the start of each note
- the burst should happen per note, not as one global wash

Why this is correct:

- NoiseLayer is a per-note source, just like the oscillators

If it sounds wrong:

- constant unrelated hiss means it is effectively acting global
- if attacks do not line up with notes, gate or envelope wiring is wrong

Operator checklist:

- NoiseLayer: behaves like a note-tracking source, not a background noise bed

Stage 11: Verify DualFilter In Finished Voices

At this point, stop using the tiny proof graph and load the shipped voices that actually use DualFilter.

Use these anchors:

- graphs/presets/dual_wavetable_pad_module_demo.json
- graphs/presets/hybrid_keys_module_demo.json
- graphs/presets/sub_air_pad_module_demo.json

Keep graphs/presets/glass_interaction_keys_module_demo.json as the control case, because it intentionally stays on plain Filter.

What you should hear:

- DualWavetablePad: the two wavetable layers feel fused into one pad body, not two independent layers stacked together
- HybridKeys: brightness has contour and focus, not just a blanket top-end boost
- SubAirPad: body and air feel integrated, with the main tone staying centered while the support layer stays supportive
- GlassInteractionKeys: still reads as the simpler single-filter comparison voice

What you should listen for when moving the main tone control:

- body and edge shift together in a musical way
- the voice stays coherent as it opens
- the filter move changes contour, not only brightness

If it sounds wrong:

- the voice splits into disconnected layers as it opens: the dual-stage filter story is not holding together
- the pad sounds like two oscillators plus two unrelated filters: the topology is no longer reading as one instrument
- the glass interaction voice suddenly feels blurred or softened in the same way as the other modules: you may be comparing against the wrong graph

Operator checklist:

- DualFilter: improves selected finished voices by shaping body and edge together without making the patch feel disconnected

Stage 12: Verify Unison and Stereo Behavior

Return to the Stage 4 or Stage 5 wavetable graph.

Test these controls:

- unison_voices
- unison_spread
- unison_output_mode
- VoiceMixer/input_layout

Start with:

- unison_voices = 1
- unison_output_mode = MonoMix
- mixer/input_layout = MonoVoices

Then test:

- unison_voices = 4
- unison_spread = moderate
- unison_output_mode = MonoMix

Then test stereo pairs:

- unison_output_mode = StereoPairs
- mixer/input_layout = StereoPairs

What you should hear:

- more width and density as unison increases

- stereo pairs should feel wider than mono mix
- the patch should stay controlled, not collapse or disappear

What you should see:

- when you use stereo pairs, both the oscillator and mixer layout must agree

Why this is correct:

- the oscillator and mixer have to match on layout or the voice path will not decode correctly

If it sounds wrong:

- metallic combing: too much spread or unstable phase behavior
- collapse or silence: unison_output_mode and input_layout do not match

Operator checklist:

- Wavetable0sc: unison changes density and width
- VoiceMixer: stereo-pair mode matches oscillator output layout correctly

Module Surface Validation

Once the operator proof path is working, validate the four shipped module instruments by ear.

HybridKeys

Load:

- graphs/presets/hybrid_keys_module_demo.json
- or graphs/presets/hybrid_keys_instrument.json

What you should hear:

- the easiest finished-voice keys sound in the package
- brightness should clearly open the top end
- body should add weight and glue, not only extra fizz

What means failure:

- brightness and body sound like the same control
- the analog support layer becomes detached instead of supportive

DualWavetablePad

Load:

- graphs/presets/dual_wavetable_pad_module_demo.json
- or graphs/presets/dual_wavetable_pad_instrument.json

What you should hear:

- one coherent pad with obvious motion and controllable brightness
- motion changes travel and animation, not only volume wobble
- blend changes layer balance without breaking pad identity

What means failure:

- the pad splits into two unrelated layers when you move blend
- motion sounds like generic tremolo or pitch wobble instead of timbral travel

SubAirPad

Load:

- `graphs/presets/sub_air_pad_module_demo.json`
- or `graphs/presets/sub_air_pad_instrument.json`

What you should hear:

- a broad pad with grounded support and audible air
- air should brighten the top without becoming detached hiss
- body should strengthen the main tone instead of only making it harsh

What means failure:

- the air layer sounds like constant background noise unrelated to note movement
- the low support feels like a detached drone

GlassInteractionKeys

Load:

- `graphs/presets/glass_interaction_keys_module_demo.json`
- or `graphs/presets/glass_interaction_instrument.json`

What you should hear:

- a glass or metallic keys voice with a readable sweet spot
- interaction should increase complexity and bite without instantly becoming unusable
- brightness and body should support the interaction story rather than fight it

What means failure:

- the interaction move goes from dull to broken with no useful middle range
- the voice loses attack clarity before it gains interesting complexity

Performance Surface Validation

The package now uses a shared live-control vocabulary across the modules. Validate those meanings by ear, not by the implementation names behind them.

- motion: should sound like wavetable travel or moving contour, not only loudness wobble
- brightness: should sound like top-end openness or tone opening, not a random macro
- air: should add breath, shimmer, or upper support without becoming detached hiss
- body: should add low-mid weight, drive, or glue without only adding harshness
- interaction: should increase carrier/modulator complexity in a musically useful way

If one module uses one of these roles and it does not mean roughly the same musical thing as the others, the surface has drifted even if the graph still technically works.

Expressive Play Validation

Use the shipped expressive example:

- `graphs/presets/expressive_glass_keys.json`

This graph uses the current shipped behavior:

- `MidiInput/aftertouch` -> `GlassInteractionKeys/aftertouch`
- `MidiInput/expression` -> `GlassInteractionKeys/expression`

- module modulation:
 - aftertouch → interaction
 - expression → brightness

What you should hear:

- aftertouch increases interaction complexity in a musically obvious way
- expression opens brightness in a controllable, audible way
- both moves stay stable while notes are held

What means failure:

- the expressive controls are technically connected but barely audible
- aftertouch only changes loudness instead of interaction character
- expression causes abrupt jumps instead of a playable brightness move

This is an interactive example, not a headless smoke fixture. The test here is whether expressive play feels musically worth using.

Asset-Backed Wavetable Validation

The package now ships factory wavetable assets and exposes builtin vs custom file-backed selection on the module surface. Validate that by ear using the shipped asset-backed content.

Good anchors:

- graphs/core/wavetable_asset_smoke.json
- graphs/presets/hybrid_keys_instrument.json
- graphs/presets/dual_wavetable_pad_instrument.json
- graphs/presets/glass_interaction_instrument.json
- graphs/presets/motion_texture_instrument.json
- graphs/presets/rooted_sub_bass_instrument.json

What you should hear:

- switching between builtin and custom wavetable sources should create a clear but intentional source change
- the instrument should still sound like the same instrument family when using a shipped asset-backed wavetable
- custom-file voices should not collapse into silence, harsh alias-like artifacts, or obviously wrong tone balance

What means failure:

- builtin and custom selection sound effectively identical when they should not
- the asset-backed version no longer reads as the role named by the instrument
- one asset-backed instrument sounds broken while the builtin equivalent sounds healthy

Instrument Library Sanity

Use the shipped instrument graphs as the final by-ear proof that the package still covers its intended playable roles.

Listen to:

- graphs/presets/hybrid_keys_instrument.json
- graphs/presets/glass_interaction_instrument.json
- graphs/presets/dual_wavetable_pad_instrument.json

- graphs/presets/sub_air_pad_instrument.json
- graphs/presets/rooted_sub_bass_instrument.json
- graphs/presets/motion_texture_instrument.json

What you should confirm:

- keys: playable and readable, not only impressive in isolation
- pads: broad and animated, but still coherent
- bass: grounded and supportive, not blurry
- interaction voice: distinctive but controllable
- motion texture: obviously motion-heavy without losing pitch identity completely

If the package only passes the tiny proof graphs but one of these instrument roles no longer sounds right, the package is still regressing in a user-facing way.

Retained Reference Graphs

Once the proof graphs are working, these shipped graphs are the next listening step:

- single_osc_motion_reference.json — the clearest small motion reference
- airy_keys.json — a clear NoiseLayer and VoiceDrive reference
- fm_glass_keys.json — the clearest interaction keys reference
- warm_dual_pad.json — the simplest broad DualFilter pad reference
- rooted_sub_bass.json — the simplest grounded sub-layer bass reference
- spectral_interaction_texture.json — the strongest interaction-texture hero patch
- expressive_glass_keys.json — the current expressive-play reference

If one of these sounds wrong, use the stages above to decide whether the problem is the oscillator path, per-note envelope path, drive placement, filter behavior, sub layer, noise layer, or interaction path.

Common False Positives

These are results that can look like success but are not actually proof:

- a raw ugly sound before VoiceMixer is okay and expected
- silence before lane inputs exist is correct
- a global beat envelope is not proof of per-note envelope behavior
- a big lush preset sounding good is not proof that a single operator is correct
- a noisy texture is not proof that NoiseLayer is per-note
- a brighter sound is not automatically proof that DualFilter is shaping contour correctly
- a connected expressive control is not proof that it is musically useful

Quick Regression Matrix

Target	Minimal proof	Expected result	Obvious failure mode
PolyVoiceAllocator	2	rough multi-note chord	one stuck note or mono-like behavior
Wavetable0sc	1, 2, 6, 7, 12	silence first, then raw poly tone, then source and motion differences	self-starts, ignores source changes, wrong modulation target
VoiceDrive	3	more body before reduction, still per-note	sounds like global post-mix distortion
VoiceMixer	4, 12	proper stereo note articulation	silence or mismatched stereo layout
EnvelopeAu	4, 5	note-by-note attacks and filter sweeps	global swell or decaying retriggers

Target	Minimal proof	Expected result	Obvious failure mode
Filter	5	per-note opening that tracks pitch	dull wrong tracking or global envelope feel
Analog0sc	8	classic simpler tone, clear waveform changes	behaves like a pre-mixed bus
Sub0sc	9	added low support, not a separate drone	detached bass or no note tracking
NoiseLayer	10	breath or attack detail per note	constant hiss or non-note-aligned bursts
DualFilter	11	selected voices feel more coherent and contour-shaped	stacked disconnected filter feeling
expressive play	expressive section	aftertouch and expression create useful live moves	technically wired but musically negligible
asset-backed voices	asset section	builtin vs custom source changes stay intentional	custom source breaks role identity

Final Yes/No Checklist

You should be able to answer yes to all of these:

- Wavetable0sc stays silent until it gets real note lanes.
- PolyVoiceAllocator produces clearly polyphonic note behavior.
- VoiceDrive adds body before reduction without turning into a global saturator.
- VoiceMixer is the point where note lanes become normal stereo audio.
- EnvelopeAu works correctly when driven from voices/gates.
- Filter behaves per-note when driven from a per-note envelope.
- Wavetable0sc families sound meaningfully different from each other.
- Wavetable0sc motion inputs change timbre rather than pitch.
- Analog0sc works as a proper per-note source.
- Sub0sc reinforces notes instead of creating a detached bass drone.
- NoiseLayer behaves like a per-note layer instead of a global hiss bed.
- DualFilter makes the selected finished voices feel more coherent, not more disconnected.
- each shipped module still reads as its intended role.
- the shared performance roles still mean the same musical thing across modules.
- aftertouch and expression create musically useful expressive moves in `expressive_glass_keys.json`.
- asset-backed instruments still sound intentional and role-correct.
- the shipped instrument library still covers playable keys, pads, bass, texture, and interaction voices convincingly.

If any answer is no, the section where that failed is the exact place to debug.